Framework de chiffrement

Travail pratique n° 1 - Programmation libre

Cours SSAD, USTHB, 2026-2027

Chiffrements de César, Hill et Playfair, attaques et protection des mots de passe, authentification des messages

Membres du groupe	Tâche
Membre 1	Chiffrement de César et ses attaques (force brute, fréquences)
Membre 2	Chiffrement de Hill et attaque à clair connu
Membre 3	Chiffrement de Playfair et analyse des digrammes
Membre 4	Attaques sur mots de passe : dictionnaire et force brute, trois cas
Membre 5	Protection des mots de passe et intégrité HMAC (partie 4)
Membre 6	Interface, tests unitaires, mesures de performance, rapport

1. Ce que nous avons réalisé

Le sujet demande un petit framework de chiffrement en quatre parties : une interface, trois chiffrements classiques, une étude des attaques sur les mots de passe avec une solution de protection, et un mécanisme d'authenticité ou d'intégrité. Tout est écrit en Python 3 avec la bibliothèque standard uniquement (Tkinter pour la fenêtre, hashlib et hmac pour le hachage). Le programme se lance avec `python app.py` et les tests avec `python tests.py`.

Fichier	Contenu
ciphers.py	César, Hill, Playfair et les attaques correspondantes
passwords.py	Les trois cas de mots de passe, attaques par dictionnaire et force brute, stockage PBKDF2, verrouillage
integrity.py	Empreinte SHA-256 et tag HMAC, scellement et vérification
app.py	Interface Tkinter à quatre onglets
partie2.py	Programme console autonome pour la partie 2 (utilise seulement ciphers.py)
tests.py	10 tests (mots de passe, intégrité) et un petit banc d'essai (python tests.py bench)
tests_partie2.py	21 tests des chiffrements

L'interface comporte quatre onglets. **Messagerie** rejoue le scénario du sujet : l'entité A saisit un message, choisit un algorithme et une clé, puis l'envoie ; le chiffré et son tag d'authentification apparaissent dans une zone « canal » modifiable à la main pour jouer l'attaquant ; l'entité B vérifie le tag puis déchiffre. **Attaques sur les chiffrements** montre ce qu'un attaquant peut faire contre chaque algorithme. **Attaques sur mots de passe** lance les attaques par dictionnaire et force brute sur les trois cas. **Protection** présente les défenses et le tableau des temps.

2. Partie 2 - Chiffrement et déchiffrement

A envoie un message à B. Les deux partagent l'algorithme et la clé. Les trois chiffrements traitent les espaces comme n'importe quel autre caractère : ils sont chiffrés eux aussi, ce qui cache la longueur des mots. Les lettres accentuées sont d'abord ramenées à leur lettre de base (é devient e). Avant d'être acceptée, chaque clé est contrôlée : une clé qui laisserait le message inchangé (décalage nul, matrice identité, clé Playfair trop courte) est refusée.

2.1 Chiffrement de César

L'alphabet compte 53 symboles : les 26 minuscules, les 26 majuscules et l'espace. Chaque symbole est décalé de n positions dans ce cycle, vers la droite ou vers la gauche. Avec des positions de 0 à 52, la formule est `c = (p + n) mod 53` pour un décalage à droite et `c = (p - n) mod 53` à gauche ; le déchiffrement applique le décalage inverse. Après z vient A, après Z vient l'espace, après l'espace revient a. La ponctuation et les chiffres sont conservés. Comme n est pris modulo 53, décaler de 56 revient à décaler de 3. Un décalage multiple de 53 ne change pas le message, le programme le refuse.

Failles de sécurité.

- Il n'existe que 53 clés (52 utiles) : tester tous les décalages prend quelques microsecondes. L'interface affiche les 53 candidats et un humain repère tout de suite la bonne ligne.

- Les fréquences des symboles survivent. Pour deviner la clé, le programme essaie chaque décalage et garde celui dont le résultat contient le plus de symboles très fréquents en français et en anglais (espace, e, t, a, o, i, n...). Cette méthode retrouve la clé sur une phrase de quelques dizaines de caractères.
- L'espace est le symbole le plus fréquent d'un texte ; le chiffrer ne le cache pas, il devient simplement le symbole le plus fréquent du chiffré.

2.2 Chiffrement de Hill

L'alphabet compte 27 symboles : A à Z et l'espace (valeurs 0 à 26). La clé est une matrice K de taille n x n à coefficients entiers modulo 27. Le texte est découpé en blocs de n symboles ; chaque bloc est un vecteur colonne p et le bloc chiffré est `c = K p mod 27`. Le déchiffrement utilise la matrice inverse, `p = K^-1 c mod 27`. L'inverse n'existe que si pgcd(det K, 27) = 1, c'est-à-dire si le déterminant n'est pas divisible par 3 ; le programme le vérifie et refuse les mauvaises clés avec un message clair. L'inverse est calculé à partir de la matrice adjointe et de l'inverse modulaire du déterminant. Si la longueur du texte n'est pas un multiple de n, on complète avec des espaces, qui sont retirés au déchiffrement. La clé peut être saisie sous forme de nombres (4,5,3,2) ou d'un mot de 4 ou 9 caractères.

Exemple détaillé, clé [[4,5],[3,2]], message HELP. Les blocs sont HE = (7,4) et LP = (11,15). K(7,4) = (48,29) = (21,2) = VC et K(11,15) = (119,63) = (11,9) = LJ, donc le chiffré est VCLJ. Cette valeur est vérifiée dans les tests.

Failles de sécurité.

- Le chiffrement est linéaire. Avec n blocs de clair connus et leurs chiffrés, K = C P^-1 mod 27 s'obtient par une seule inversion de matrice. Notre onglet d'attaque le fait pour une clé 2x2 à partir de 4 caractères connus et retrouve la clé exacte.
- Une clé 2x2 n'a qu'un nombre limité de matrices inversibles, donc la force brute est facile pour les petites tailles.
- Deux blocs identiques donnent deux chiffrés identiques, ce qui révèle les répétitions (fonctionnement de type ECB).
- Il n'y a pas de diffusion entre blocs : modifier un bloc n'affecte jamais les autres.

2.3 Chiffrement de Playfair

Le problème de I et J. Le Playfair classique utilise un carré 5x5 de 25 cases pour 26 lettres, et il faut donc confondre J avec I. Au déchiffrement, JAVA revient en IAVA : le message est abîmé et une même clé donne le même chiffré pour deux lettres différentes.

Notre solution. Nous remplaçons le carré 5x5 par un carré **7x7 de 49 cases**. Il contient les 26 lettres (I et J séparés), les 10 chiffres, l'espace, 12 signes de ponctuation (. , ! ? ' - : ; ( ) /) et le caractère # réservé au bourrage. Le carré est construit avec le mot clé (symboles répétés supprimés), suivi du reste des 49 symboles dans un ordre fixe. Il n'y a plus aucune fusion, les espaces et les chiffres sont chiffrés, et la clé peut contenir des chiffres.

Bourrage sans ambiguïté. Dans le Playfair classique, le X sert à séparer deux lettres identiques et à compléter la dernière paire, ce qui oblige à deviner s'il faut retirer un X au déchiffrement (MAX devenait MA, AXA devenait AA). Ici le bourrage utilise # qui ne peut pas apparaître dans un message, car il est retiré à l'entrée. Au déchiffrement, on supprime tous les # : le message d'origine

revient exactement.

Règles du chiffrement, inchangées : le message est découpé en paires ; deux symboles identiques dans une paire reçoivent un # intercalé ; un symbole seul à la fin est complété par #. Pour chaque paire : même ligne, on prend le symbole à droite ; même colonne, celui du dessous ; sinon on échange les colonnes pour former le rectangle. Le déchiffrement va vers la gauche et vers le haut.

Failles de sécurité.

- Il travaille sur des digrammes, soit 49 x 48 = 2 352 paires possibles au lieu de 600, ce qui rend l'analyse de fréquence beaucoup plus difficile qu'avec le carré 5x5. Elle reste possible avec assez de texte, et la fonction de comptage affiche les paires les plus répétées.
- Une paire et son inverse sont chiffrées en paires inverses (AB -> XY, BA -> YX), une régularité utile à l'attaquant.
- Un symbole n'est jamais apparié à lui-même ni chiffré en lui-même, ce qui réduit les hypothèses.
- L'escalade de colline (hill climbing) et le recuit simulé le cassent sur quelques centaines de caractères.

3. Partie 3 - Mots de passe

Pour des expériences réalistes, le programme fait ce que permet une base de données volée : il ne garde que l'empreinte SHA-256 du mot de passe, et l'attaquant essaie des candidats, les hache et compare. Trois cas étaient demandés.

Cas	Taille de l'alphabet	Longueur	Espace de recherche
Binaire	2 (0,1)	3	2^3 = 8
Chiffres 1-4	4	4	4^4 = 256
Ensemble a-z A-Z 0-9 + * ...	74	3 (démo)	74^3 = 405224

L'ensemble complet utilisé compte 26 minuscules + 26 majuscules + 10 chiffres + 13 symboles (+ * - _ @ # $! ? % & =), soit 74 caractères. Le sujet dit « etc. », ce choix est donc le nôtre. La longueur du troisième cas est réglable dans l'interface (1 à 4) ; nous prenons 3 dans le rapport car 4 caractères représentent déjà environ 29 millions d'essais.

3.1 Les attaques

Attaque par dictionnaire. L'attaquant hache une liste de mots de passe probables (la nôtre compte une cinquantaine d'entrées : mots de passe courants, courtes suites de chiffres, quelques mots locaux) et compare. Le coût est au plus la taille de la liste, c'est donc la première chose à essayer. Mesure : « qwerty » est trouvé après 4 essais en 7 µs. Un mot de passe aléatoire comme « Xk9+mQ » n'est pas dans la liste et l'attaque s'arrête après 47 essais sans résultat.

Attaque par force brute. Toutes les combinaisons de l'alphabet sont générées avec `itertools.product` et testées. Elle réussit toujours, seule la durée change.

Temps minimal. Pour chaque cas, la stratégie combinée est : dictionnaire d'abord (peu coûteux, couvre les choix humains faibles), puis recherche exhaustive sur l'alphabet exact et la longueur exacte, ce qui donne le plus petit espace possible. Pour les deux petits cas, l'espace entier est plus petit que le coût du dictionnaire, la recherche exhaustive est donc instantanée. Dans la boucle

interne, nous comparons aussi les octets bruts de l'empreinte plutôt que des chaînes hexadécimales, ce qui est plus rapide.

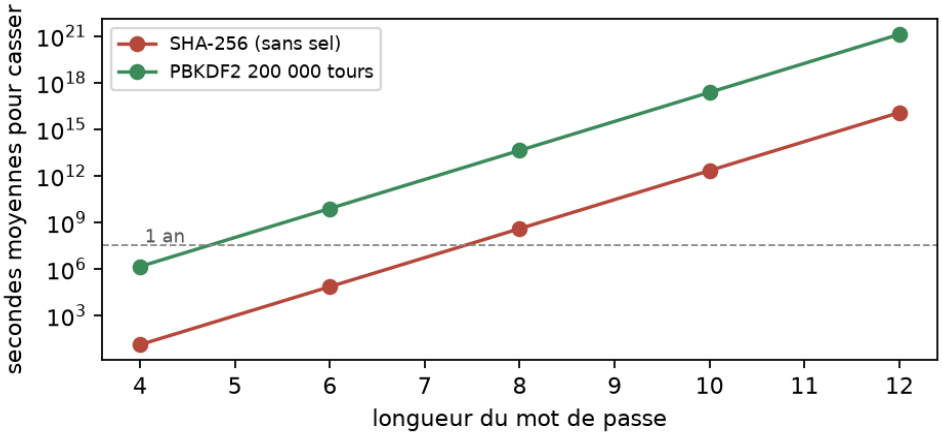
Cas	Espace	Essais (pire cas)	Temps
3 car., 0/1	8	8	48 µs
4 car., 1-4	256	256	256 µs
3 car., ensemble complet	405224	405224	372.1 ms

La machine utilisée atteint environ 1150949 essais SHA-256 par seconde en Python (une carte graphique en fait des milliards par seconde, les vrais attaquants sont donc des milliers de fois plus rapides).

3.2 Protection

Les attaques ci-dessus réussissent parce que le hachage est rapide et que l'alphabet ou la longueur sont petits. Notre proposition a quatre couches, toutes implémentées dans passwords.py.

- **Hachage salé et lent.** Les mots de passe sont stockés avec PBKDF2-HMAC-SHA256, un sel aléatoire de 16 octets par utilisateur et 200 000 itérations. Le sel neutralise les tables précalculées et fait que deux utilisateurs ayant le même mot de passe ont des empreintes différentes. Le nombre d'itérations rend chaque essai coûteux : une vérification coûte environ 94.5 ms ici, l'attaquant passe donc d'environ 1150949 essais par seconde à environ 10, soit un ralentissement d'environ 108780x.
- **Limitation de débit et verrouillage.** La classe LoginGuard verrouille le compte 30 secondes après 3 échecs. Un attaquant en ligne n'a alors que 6 essais par minute au lieu de milliers. Les tests vérifient qu'un bon mot de passe est refusé pendant le verrouillage.
- **Politique de mot de passe.** strength_problems refuse les mots de passe de moins de 10 caractères, ceux de la liste courante, et ceux sans minuscule, majuscule, chiffre ou symbole. Cela neutralise le dictionnaire et fait croître exponentiellement l'espace de la force brute.
- **Comparaison en temps constant.** hmac.compare_digest est utilisé pour que le temps de réponse ne révèle pas combien d'octets correspondent.



Le graphique montre le temps moyen pour casser un mot de passe de l'ensemble de 74 caractères selon sa longueur. Sans sel et avec un hachage rapide, 8 caractères tiennent environ 12.4 ans sur notre matériel ; avec le hachage lent, le même mot de passe tient 1.3e+06 ans. Chaque caractère ajouté multiplie le travail par 74.

Remarque importante : les deux petits cas (8 et 256 combinaisons) ne peuvent pas être protégés par le hachage, car n'importe quel attaquant peut tout essayer instantanément même avec un hachage lent (256 x 0,14 s font environ 36 secondes). La seule défense est le verrouillage et, bien sûr, ne pas utiliser de tels mots de passe.

4. Partie 4 - Authenticité et intégrité

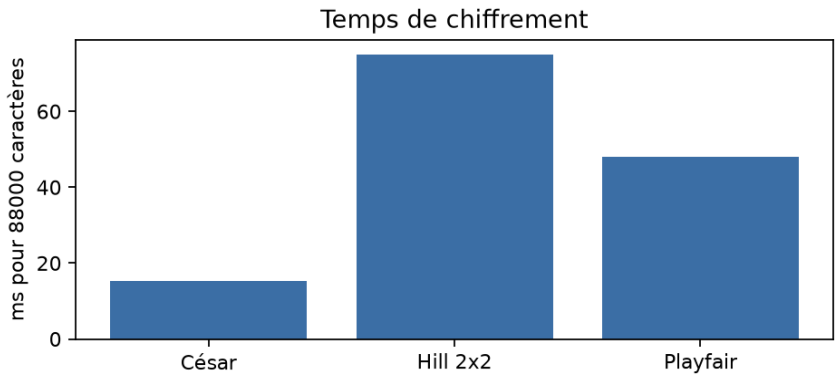
Nous avons choisi un objectif : **permettre à B de détecter toute modification du message et de savoir qu'il vient de quelqu'un qui possède le secret partagé.** L'outil est un tag HMAC-SHA256 calculé sur le chiffré avec un secret partagé par A et B (chiffrer puis authentifier).

- A calcule tag = HMAC(secret, chiffré) et envoie les deux.
- B recalcule le tag sur ce qu'il a reçu et compare en temps constant. S'il diffère, le message est rejeté avant tout déchiffrement.
- Dans l'onglet Messagerie, le bouton « Changer une lettre » simule un attaquant qui modifie une lettre sur le canal ; B affiche alors « intégrité non vérifiée ». Un mauvais secret donne le même résultat, ce qui illustre l'authenticité.

Sans cette couche, les chiffrements classiques n'offrent aucune intégrité : en César, changer une lettre du chiffré modifie silencieusement la lettre correspondante du clair. Notons aussi qu'un HMAC prouve que l'émetteur connaît le secret mais n'apporte pas la non-répudiation ; pour cela il faudrait des signatures numériques.

5. Tests et performances

Les 31 tests unitaires (21 dans tests_partie2.py, 10 dans tests.py) réussissent tous. Ils couvrent : les allers-retours de César dans les deux sens pour des décalages de 0 à 119, les attaques connues sur César, l'exemple de Hill calculé à la main (clé 4,5,3,2, HELP -> VCLJ), 40 clés Hill aléatoires de taille 2 et 3 modulo 27, le rejet d'une clé non inversible, l'attaque à clair connu, les allers-retours Playfair (I et J, espaces, chiffres, accents, X en fin de mot, 200 textes aléatoires), les messages de longueur impaire, le refus des clés faibles, les tailles d'espaces de recherche, la force brute sur chacun des 8 mots de passe binaires, le succès et l'échec du dictionnaire, la vérification PBKDF2 et l'unicité du sel, le temps de verrouillage, et la détection de modification par HMAC.



Algorithme	Temps pour 88000 caractères
César	15.4 ms
Hill 2x2	75.0 ms

Playfair	48.0 ms

Les trois chiffrements traitent environ 90 ko de texte en moins d'un dixième de seconde, la vitesse n'est donc pas un problème. Hill est le plus lent par caractère à cause des produits de matrices, César le plus rapide. La vraie différence entre eux n'est pas la vitesse mais la résistance, vue en section 2.

6. Répartition du travail

Membre	Fichiers / sections	Livrable
1	ciphers.py (caesar, caesar_bruteforce, caesar_frequency_guess)	César dans les deux sens, attaques, section 2.1
2	ciphers.py (fonctions Hill, inverse de matrice mod 26)	Chiffrement de Hill, attaque à clair connu, section 2.2
3	ciphers.py (fonctions Playfair, digram_counts)	Playfair 7x7 (I/J séparés), bourrage #, section 2.3
4	passwords.py (espace de recherche, dictionnaire, force brute)	Les trois cas, mesures de temps, section 3.1
5	passwords.py (PBKDF2, LoginGuard), integrity.py	Protection, HMAC, sections 3.2 et 4
6	app.py, tests.py, rapport	Interface, 31 tests, mesures, sections 1 et 5

7. Conclusion

Les trois chiffrements fonctionnent, font l'aller-retour correctement et sont vérifiés sur des exemples publiés, mais chacun tombe devant une attaque connue que notre interface démontre : recherche exhaustive de clé ou fréquences pour César, clair connu pour Hill, statistiques de digrammes pour Playfair. Ils servent à apprendre, pas à protéger de vraies données. Pour les mots de passe, les petits alphabets et les longueurs courtes sont cassés instantanément quoi que l'on fasse, et les plus grands ne sont sûrs qu'avec salage, hachage lent, politique de mots de passe et verrouillage. Enfin, la couche HMAC montre que la confidentialité seule ne suffit pas : l'intégrité doit être vérifiée séparément.